

## Resenha – Design Pattern Facade

O Facade é um padrão de projeto estrutural catalogado pelo Refactoring.Guru que tem como objetivo fornecer uma interface simplificada para um conjunto complexo de classes, bibliotecas ou frameworks. Ele pertence à categoria dos padrões estruturais e resolve um problema bastante comum no desenvolvimento de software: o acoplamento excessivo entre o código cliente e os detalhes internos de um subsistema.

O problema que motiva o uso desse padrão é direto. Conforme um subsistema cresce, o código que o utiliza precisa conhecer cada vez mais detalhes de implementação, como inicializar objetos na ordem certa, gerenciar dependências e chamar métodos em sequências específicas. Esse conhecimento espalhado pelo código cliente dificulta a manutenção e compromete a legibilidade do sistema.

A solução proposta pelo padrão é criar uma classe intermediária, a fachada, que encapsula toda essa complexidade e expõe ao cliente apenas os métodos realmente necessários. A fachada não substitui o subsistema nem elimina sua funcionalidade. Ela oferece um atalho para os casos de uso mais comuns, mantendo o subsistema acessível diretamente para quem precisar de mais controle. Vale ressaltar que as classes do subsistema não têm conhecimento da existência da fachada, elas continuam operando de forma independente entre si.

Para ilustrar o padrão em Java, imagine um sistema de envio de e-mails que depende internamente de várias classes: `SmtplibConnection`, `EmailValidator`, `EmailTemplateEngine` e `EmailLogger`. Sem uma fachada, o código cliente teria que instanciar e orquestrar todas essas classes manualmente a cada envio. Com o padrão aplicado, uma classe `EmailServiceFacade` com um único método `enviar` (`String` destinatário, `String` assunto, `String` corpo) centraliza toda essa lógica. O cliente passa a interagir apenas com esse método, sem saber nada sobre como a conexão SMTP é estabelecida, como o template é processado ou como os logs são gravados. Se a biblioteca de envio mudar, a alteração fica restrita à implementação interna da fachada, sem afetar o código cliente.

O Refactoring.Guru aponta dois cenários principais para usar o padrão. O primeiro é quando o cliente precisa de só uma parte das funcionalidades de um subsistema complexo, situação em que criar a fachada quase sempre vale a pena. O segundo é na organização de sistemas em camadas, onde fachadas funcionam como pontos de entrada bem definidos para cada camada, reduzindo o acoplamento entre elas.

O padrão tem uma vantagem clara: o código cliente fica protegido das mudanças internas do subsistema. Por outro lado, existe um risco que merece atenção. A fachada pode acumular responsabilidades ao longo do tempo e virar o que a literatura chama de "objeto deus", se tornando ela mesma um ponto de acoplamento indesejado. Quando isso acontece, a recomendação é extrair parte do comportamento para fachadas adicionais mais especializadas.

Por fim, é útil entender as diferenças em relação a padrões parecidos. Diferente do `Adapter`, que adapta uma interface existente para outra, o `Facade` cria uma interface

nova e simplificada para um subsistema inteiro. Em relação ao Mediator, apesar dos dois organizarem a comunicação entre classes, o Facade não centraliza a comunicação interna do subsistema, os componentes internos continuam podendo se comunicar diretamente entre si. O Facade, portanto, é uma solução simples e bastante aplicável para lidar com complexidade em sistemas de software.